In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [11]:
```
# %pip install opencv-python
```

In [ ]:
```
#
# Design n-Queens matrix having first Queen placed. Use backtracking to plac
# remaining Queens to generate the final n-queen's matrix.
```

In [24]:
```
# N-Queens using Backtracking

def is_safe(board, row, col, n):

    for i in range(row):
        if board[i][col] == 1:        # checks if it is empty
            return False

    i, j = row - 1, col - 1      #i moves upward and j left means left diago
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i, j = row - 1, col + 1     #upper right dialgonal checking
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True    #if no conflicts in upper or same is found return true


def solve(board, row, n):

    if row == n:
        return True

    for col in range(n):
        if is_safe(board, row, col, n):       #add a queen at certain positi

            board[row][col] = 1

            if solve(board, row + 1, n):   #add a queen in the next row if
                                           #there is a queen return true and
                return True

            board[row][col] = 0

    return False    #returns false when no safe column for current row is fo


# Main program
```

```
n = int(input("Enter number of queens: "))

if n <= 0:
    print("Enter a positive number.")
else:
    board = [[0] * n for i in range(n)]

    first_col = int(input("Enter first queen column (1 to n): ")) - 1    #-1


    if first_col < 0 or first_col >= n:
        print("Invalid column.")
    else:
        board[0][first_col] = 1

        if solve(board, 1, n):     #backtracking from row one
            print("Final N-Queens Matrix:")
            for row in board:
                print(row)        #display complete matrix
        else:
            print("No solution possible.")
```

```
Final N-Queens Matrix:
[0, 1, 0, 0]
[0, 0, 0, 1]
[1, 0, 0, 0]
[0, 0, 1, 0]
```

In [ ]: